

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (AI)

DEEP LEARNING LAB RECORD

LAB - 01

Student Details

Name: K. Charan Teja

Register Number: CH.SC.U4AIE24032

Course Name: Deep Learning

Course Code: 22AIE304

Experiment Title

Perceptron Learning Implementation

```
In [1]: import numpy as np
import sklearn

print("NumPy:", np.__version__)
print("Scikit-Learn:", sklearn.__version__)
```

NumPy: 1.20.3
Scikit-Learn: 0.24.2

```
In [2]: from sklearn.datasets import load_digits

digits = load_digits()

print(digits.data.shape)
print(digits.target.shape)
```

(1797, 64)
(1797,)

```
In [3]: from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score
```

```

digits = load_digits()

X_train, X_test, y_train, y_test = train_test_split(
    digits.data,
    digits.target,
    test_size=0.2,
    random_state=42
)

model = MLPClassifier(
    hidden_layer_sizes=(100,),
    max_iter=1000
)

model.fit(X_train, y_train)

pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, pred))

```

Accuracy: 0.9722222222222222

```

In [4]: import matplotlib.pyplot as plt
        from sklearn.datasets import load_digits

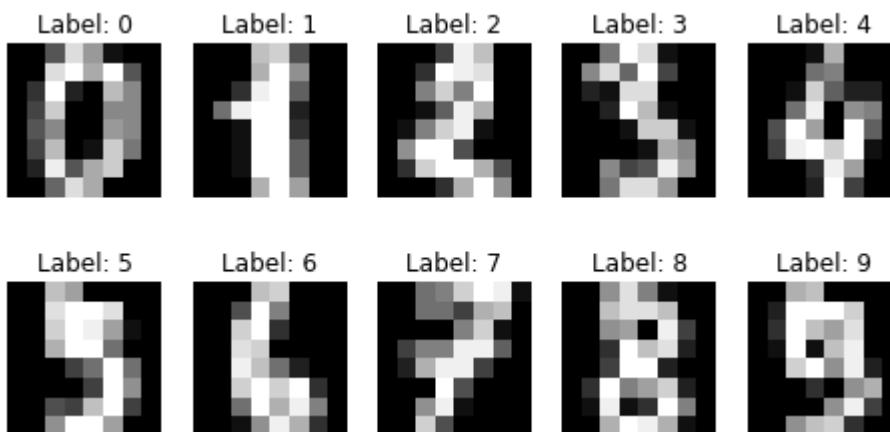
        digits = load_digits()

        plt.figure(figsize=(8,4))

        for i in range(10):
            plt.subplot(2,5,i+1)
            plt.imshow(digits.images[i], cmap='gray')
            plt.title(f"Label: {digits.target[i]}")
            plt.axis('off')

        plt.show()

```



```

In [5]: import numpy as np

        # AND Gate Dataset
        X = np.array([
            [0,0],
            [0,1],
            [1,0],
            [1,1]
        ])

```

```

y = np.array([0,0,0,1])

# Initialize weights and bias
weights = np.zeros(2)
bias = 0
learning_rate = 0.1

# Training
for epoch in range(10):
    for i in range(len(X)):
        z = np.dot(X[i], weights) + bias
        pred = 1 if z >= 0 else 0

        error = y[i] - pred

        weights += learning_rate * error * X[i]
        bias += learning_rate * error

print("AND Gate Predictions")
for x in X:
    z = np.dot(x, weights) + bias
    pred = 1 if z >= 0 else 0
    print(x, "->", pred)

```

```

AND Gate Predictions
[0 0] -> 0
[0 1] -> 0
[1 0] -> 0
[1 1] -> 1

```

In [6]: `import numpy as np`

```

# OR Gate Dataset
X = np.array([
    [0,0],
    [0,1],
    [1,0],
    [1,1]
])

y = np.array([0,1,1,1])

weights = np.zeros(2)
bias = 0
learning_rate = 0.1

# Training
for epoch in range(10):
    for i in range(len(X)):
        z = np.dot(X[i], weights) + bias
        pred = 1 if z >= 0 else 0

        error = y[i] - pred

        weights += learning_rate * error * X[i]
        bias += learning_rate * error

print("OR Gate Predictions")
for x in X:

```

```

z = np.dot(x, weights) + bias
pred = 1 if z >= 0 else 0
print(x, "->", pred)

```

OR Gate Predictions

```

[0 0] -> 0
[0 1] -> 1
[1 0] -> 1
[1 1] -> 1

```

In [7]: `from sklearn.neural_network import MLPClassifier`

```

# XOR Dataset
X = [
    [0,0],
    [0,1],
    [1,0],
    [1,1]
]

y = [0,1,1,0]

model = MLPClassifier(
    hidden_layer_sizes=(8,),
    activation='tanh',
    solver='lbfgs',
    max_iter=10000,
    random_state=1
)

model.fit(X, y)

pred = model.predict(X)

print("XOR Gate Predictions")
for i in range(len(X)):
    print(X[i], "->", pred[i])

print("Accuracy:", model.score(X, y))

```

XOR Gate Predictions

```

[0, 0] -> 0
[0, 1] -> 1
[1, 0] -> 1
[1, 1] -> 0
Accuracy: 1.0

```

In [8]: `from sklearn.datasets import load_digits`
`from sklearn.model_selection import train_test_split`
`from sklearn.neural_network import MLPClassifier`
`from sklearn.metrics import accuracy_score`

```

# Load dataset
digits = load_digits()

print("Dataset Shape:", digits.data.shape)
print("Labels Shape:", digits.target.shape)

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    digits.data,

```

```

    digits.target,
    test_size=0.2,
    random_state=42
)

# Create Neural Network
model = MLPClassifier(
    hidden_layer_sizes=(100,),
    max_iter=1000,
    random_state=42
)

# Train
model.fit(X_train, y_train)

# Predict
pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, pred)

print("Test Accuracy:", accuracy)

```

Dataset Shape: (1797, 64)

Labels Shape: (1797,)

Test Accuracy: 0.9833333333333333

In [9]:

```

import matplotlib.pyplot as plt
from sklearn.datasets import load_digits

digits = load_digits()

plt.figure(figsize=(8,4))

for i in range(10):
    plt.subplot(2,5,i+1)
    plt.imshow(digits.images[i], cmap='gray')
    plt.title(f"Label: {digits.target[i]}")
    plt.axis('off')

plt.tight_layout()
plt.show()

```

